

TRAVAUX PRATIQUES

# SmartGrid

Smart Grid - Big Data et Intelligence Artificielle

## Projet SmartGrid — Rapport complet

*Entrepôt de données, prévision de consommation et détection d'anomalies*

**Etudiant**

Ahmed Benahmed — Entrepôt de  
données  
Elwalid Aboulaakoul — Intelli-  
gence artificielle

**Encadrement**

Pr. Hrimech

**Date du TP**

1 juin 2026

**Annee universitaire**

2025/2026

# Vue d'ensemble du projet

Le projet SmartGrid met en place une chaîne complète pour observer la consommation électrique de compteurs intelligents, anticiper la demande et signaler les comportements anormaux. Les compteurs produisent des séries temporelles volumineuses et hétérogènes; le projet les transforme en un flux exploitable: ingestion, stockage temporel, modélisation analytique, prévision, détection d'écarts et visualisation opérationnelle.

Le système s'articule autour de deux contributions complémentaires:

| Contribution                                           | Périmètre                                                                                                                                                                                                                |
|--------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Entrepôt de données</b> — Ahmed Benahmed            | Normalisation multi-source vers un schéma commun, modélisation dimensionnelle en étoile / galaxie, stockage TimescaleDB (hypertables, agrégats continus), couches médaillon et stratégie de déploiement progressif.      |
| <b>Intelligence artificielle</b> — Elwalid Aboulaakoul | Préparation des séries, benchmark de prévision (SeasonalNaive, Prophet, LightGBM), détection d'anomalies par résidu médiane/MAD, baseline LSTM Autoencoder, évaluation précision/rapport et démonstration reproductible. |

La chaîne de bout en bout relie ces deux parties: les mesures arrivent par Kafka ou par chargement batch, sont normalisées et stockées dans l'entrepôt, servent de base aux modèles de prévision, puis la détection compare la valeur observée à la valeur attendue avant restitution dans Grafana et supervision par Prometheus.

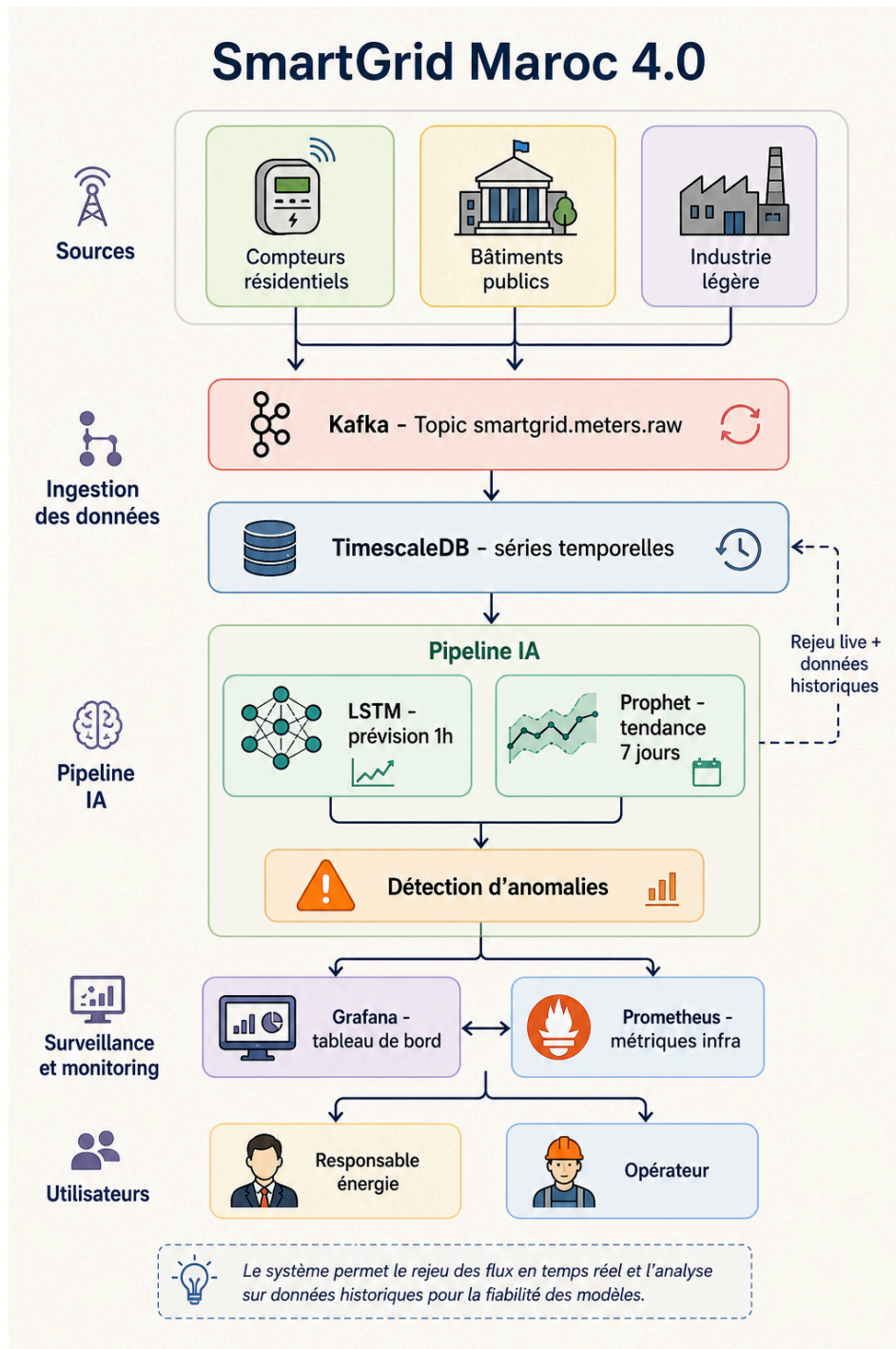


Figure 1: Architecture globale du projet: sources de mesure, ingestion Kafka, entrepôt TimescaleDB, couche IA (Prophet / LightGBM / LSTM Autoencoder), détection d'anomalies, puis supervision Grafana et Prometheus.

Les données suivent un découpage médaillon: bronze (mesures brutes), argent (faits nettoyés et dimensions conformes), or (agrégats, prévisions et anomalies exposés au tableau de bord). Ce découpage relie directement la partie entrepôt et la partie IA.

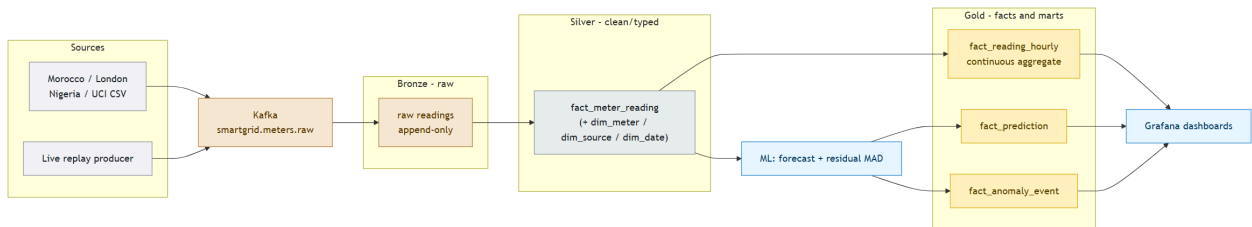


Figure 2: Flux médaillon reliant les deux parties: Kafka et le batch alimentent le bronze, le silver matérialise les faits avec leurs dimensions, l'or expose agrégats, prévisions et anomalies consommés par le ML et Grafana.

Toutes les sources sont ramenées au même format canonique, ce qui découple les modèles et le tableau de bord de la forme originale de chaque fichier:

`time, meter_id, kwh, is_anomaly, source`

Le reste du rapport détaille les deux parties: la Partie I présente la conception de l'entrepôt de données; la Partie II présente les algorithmes IA de prévision et de détection d'anomalies.

## Partie I — Conception de l'entrepôt de données

---

## Schéma en étoile / galaxie

La partie entrepôt s'appuie sur des faits temporels et des dimensions conformes partagées par tous les faits. Trois faits et quatre dimensions partagées constituent une constellation de faits (galaxie), variante multi-faits du schéma en étoile. Ce n'est pas un schéma en flocon: les dimensions ne sont pas normalisées en hiérarchies, ce qui réduit les jointures pour une charge dominée par la lecture.

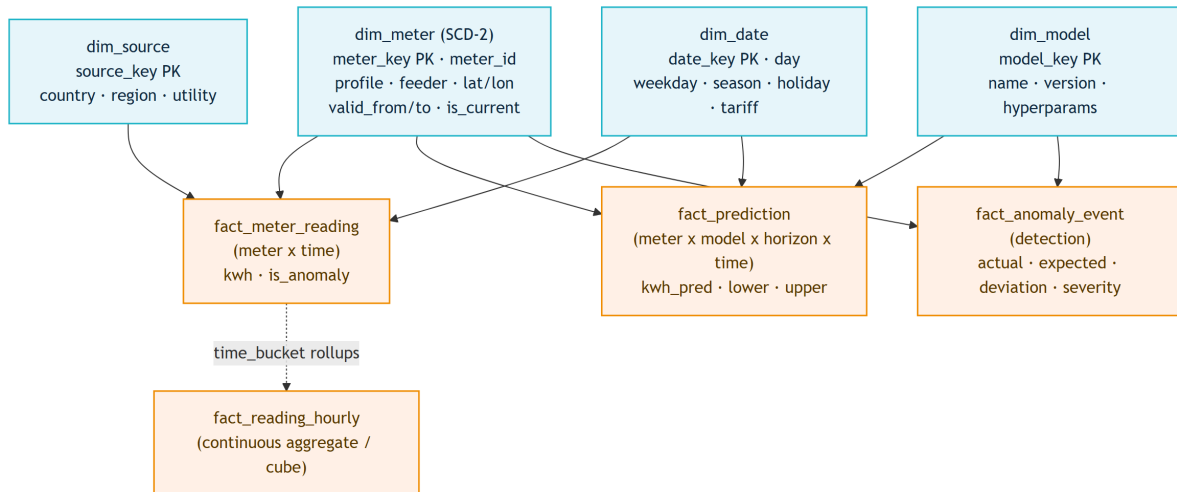


Figure 3: Schéma en étoile / galaxie. Les dimensions conformes (bleu) `dim_meter`, `dim_source`, `dim_date`, `dim_model` sont partagées par les trois tables de faits (orange). Les agrégats continus dérivent du fait de lectures et jouent le rôle de cube.

Le diagramme entité-association détaille les clés et colonnes: chaque fait référence les dimensions par clé de substitution, ce qui découple les identifiants métier de la modélisation et permet l'historisation.

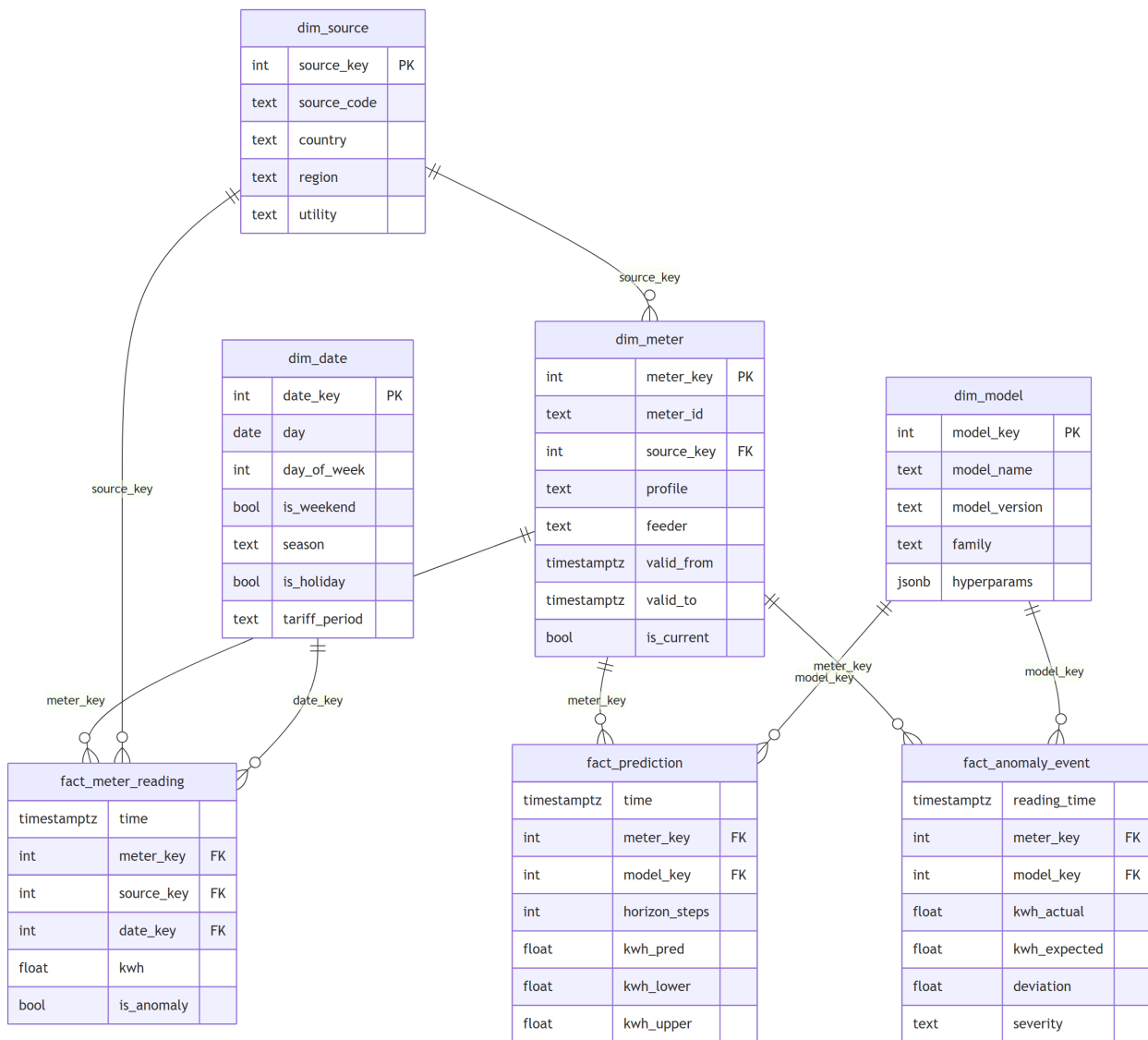


Figure 4: Modèle entité-association détaillé. Les clés de substitution (\*\_key) relient les faits aux dimensions; dim\_meter est historisée (SCD-2) via valid\_from/valid\_to/is\_current.

## Dimensions conformes

| Dimension         | Rôle et attributs clés                                                                                                                                     |
|-------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| dim_meter (SCD-2) | Clé de substitution meter_key, clé naturelle meter_id, profil, départ (feeder), localisation, lat/lon, fenêtre de validité valid_from/valid_to/is_current. |
| dim_source        | Origine du jeu de données: source_code, pays, région, opérateur (disco).                                                                                   |
| dim_date          | Calendrier: jour, jour de semaine, week-end, saison, jour férié, période tarifaire.                                                                        |
| dim_model         | Référentiel des modèles: nom, version, famille, hyperparamètres (jsonb), identifiant d'entraînement.                                                       |

## Tables de faits

| Fait (hypertable)  | Grain et mesures                                                                            |
|--------------------|---------------------------------------------------------------------------------------------|
| fact_meter_reading | Grain: compteur × instant. Mesures: kwh, is_anomaly. Index unique (meter_key, time).        |
| fact_prediction    | Grain: compteur × modèle × horizon × instant. Mesures: kwh_pred, kwh_lower, kwh_upper.      |
| fact_anomaly_event | Grain: une détection. Mesures: kwh_actual, kwh_expected, deviation, severity, anomaly_type. |

La couche physique TimescaleDB est exploitée pleinement: hypertables sur chaque fait (chunks de 7 jours), agrégats continus (15 min / horaire / journalier), compression native des chunks anciens et politique de rétention.

## Étoile ou flocon ?

**Décision: schéma en étoile.** Les dimensions du SmartGrid ont une faible cardinalité et la charge est dominée par la lecture (tableaux de bord Grafana et extraction de variables ML). Des dimensions dénormalisées réduisent les jointures et accélèrent les lectures.

**Flocon rejeté.** Normaliser les dimensions en hiérarchies (compteur → localisation → ville → région) n'est rentable que pour des dimensions très volumineuses ou redondantes — ce n'est pas le cas ici, et le flocon ajouterait du coût de jointure pour un gain de stockage négligeable.

## Déploiement progressif (sans interruption)

Le déploiement préserve la continuité de service à chaque étape: le modèle est construit dans un schéma dw dédié, les premières phases sont purement additives, et les lectures ne basculent qu'une fois la parité vérifiée, avec des vues de compatibilité aux colonnes identiques pour les consommateurs existants.

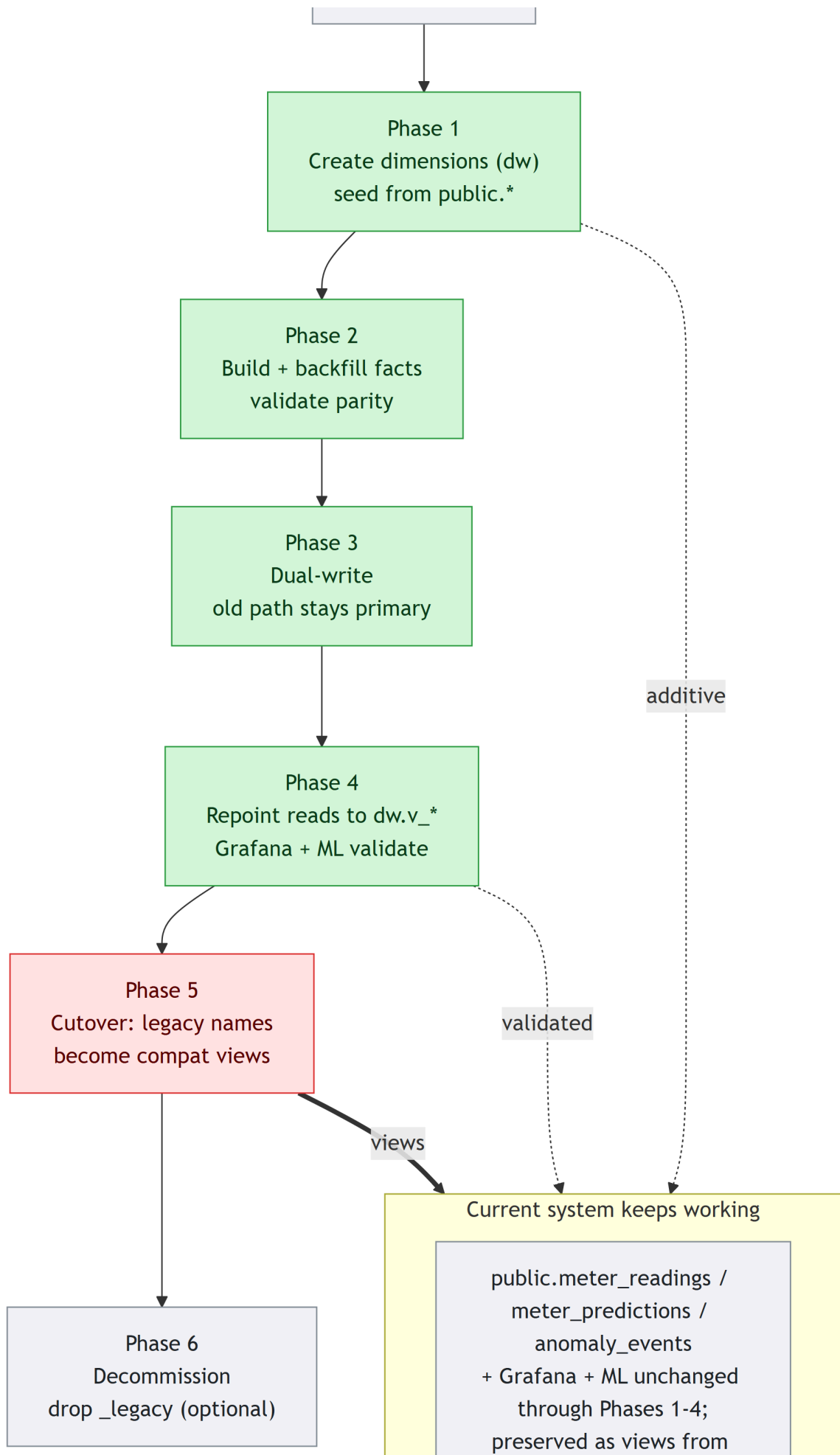


Figure 5: Déploiement en sept phases. Vert = additif (aucun impact), rouge = bascule. Le service reste opérationnel pendant les phases additives, puis les consommateurs sont préservés sous forme de vues lors de la bascule.



# Sources de données et préparation

Le projet documente quatre familles de données: Morocco High-Resolution Smart Meters (contexte marocain, conversion possible depuis l'intensité vers le kWh), London Smart Meters (profils résidentiels demi-horaires), UCI Household Power (jeu léger pour tests rapides) et Nigeria Smart Meter (validation africaine complémentaire). Toutes sont ramenées au schéma canonique commun.

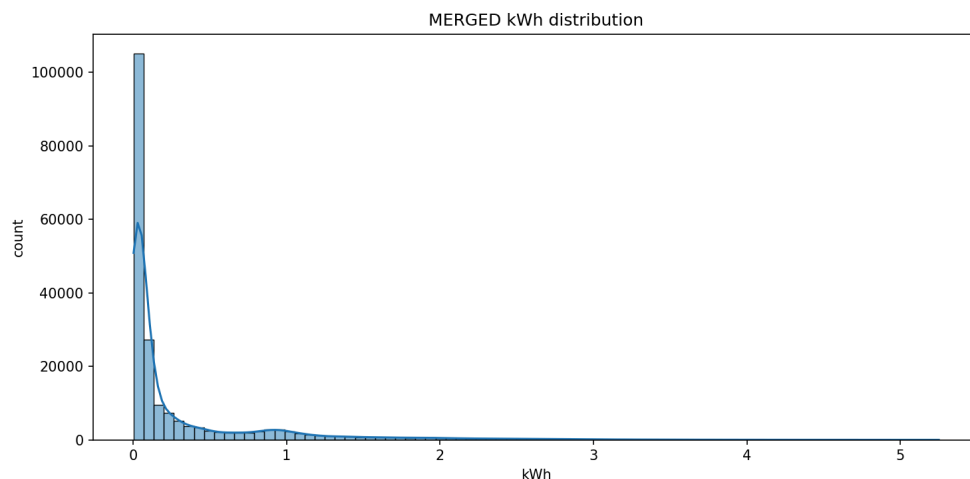


Figure 6: Distribution des consommations dans le jeu préparé. Une distribution très étalée indique que le détecteur doit comparer une mesure à son contexte local plutôt qu'à une moyenne globale unique.

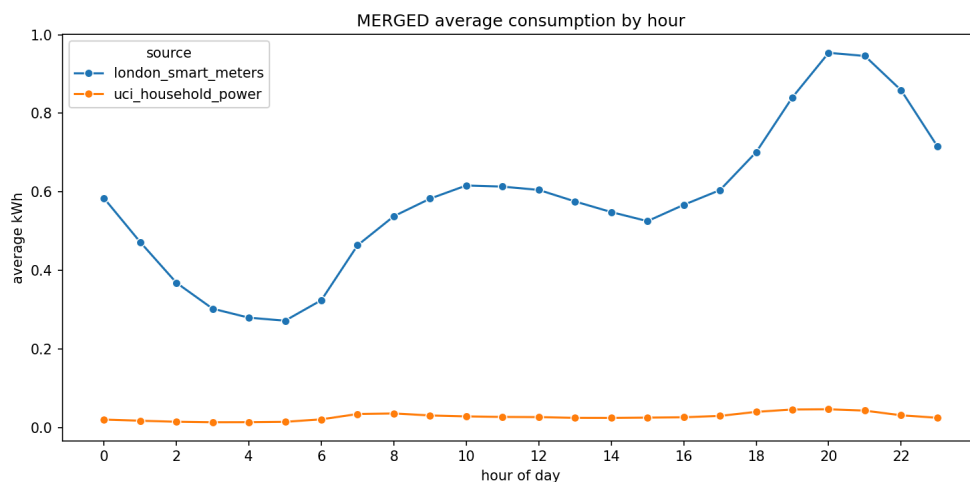


Figure 7: Profil horaire moyen par source. Les pics matin/soir et les creux nocturnes donnent le contexte attendu de consommation.

# Pipeline Machine Learning

Le pipeline ML suit la chronologie d'une série temporelle: chaque source est nettoyée, convertie en kWh si nécessaire, puis ramenée à une cadence commune de 30 minutes. La prévision produit une valeur attendue; la détection mesure l'écart entre cette valeur attendue et la valeur observée.

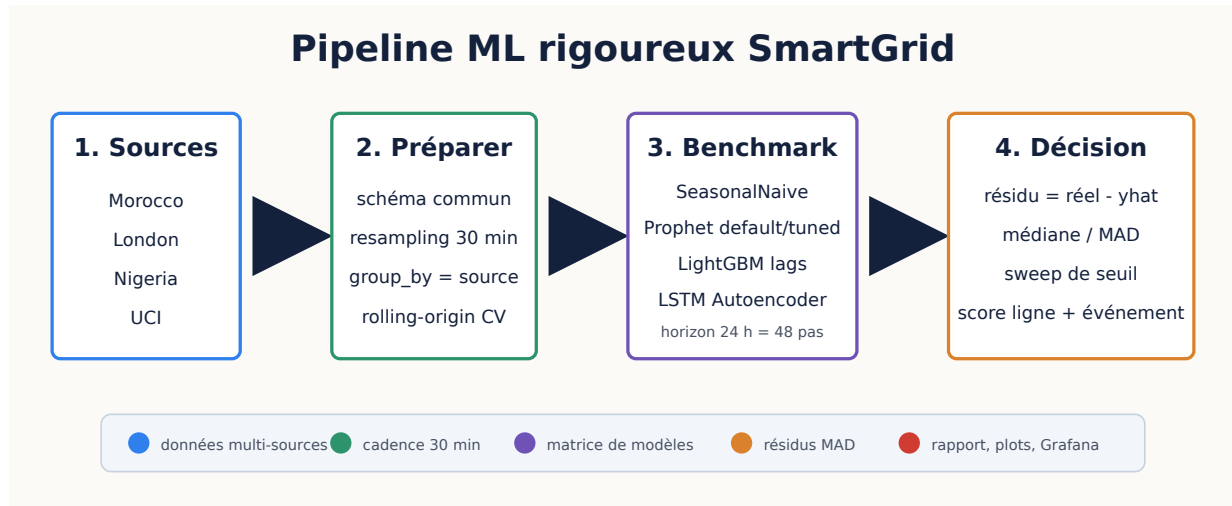


Figure 8: Pipeline ML: sources, prétraitement à 30 minutes, benchmark de prévision, calcul du résidu, sélection du seuil MAD et restitution des anomalies dans Grafana.

## Benchmark de prévision

Le benchmark utilise quatre sources naturelles et cinq folds rolling-origin sur un horizon de 24 heures (48 pas). La métrique principale est WAPE, plus stable que MAPE lorsque la consommation contient des valeurs proches de zéro.

| Modèle          | Famille  | Statut   | WAPE 1 pas | WAPE 24h |
|-----------------|----------|----------|------------|----------|
| SeasonalNaive   | forecast | complété | 0.0971     | 0.1259   |
| Prophet default | forecast | complété | 0.0594     | 2.7628   |
| Prophet tuned   | forecast | complété | 0.0597     | 1.2224   |
| LightGBM lags   | forecast | complété | 0.1148     | 0.0707   |

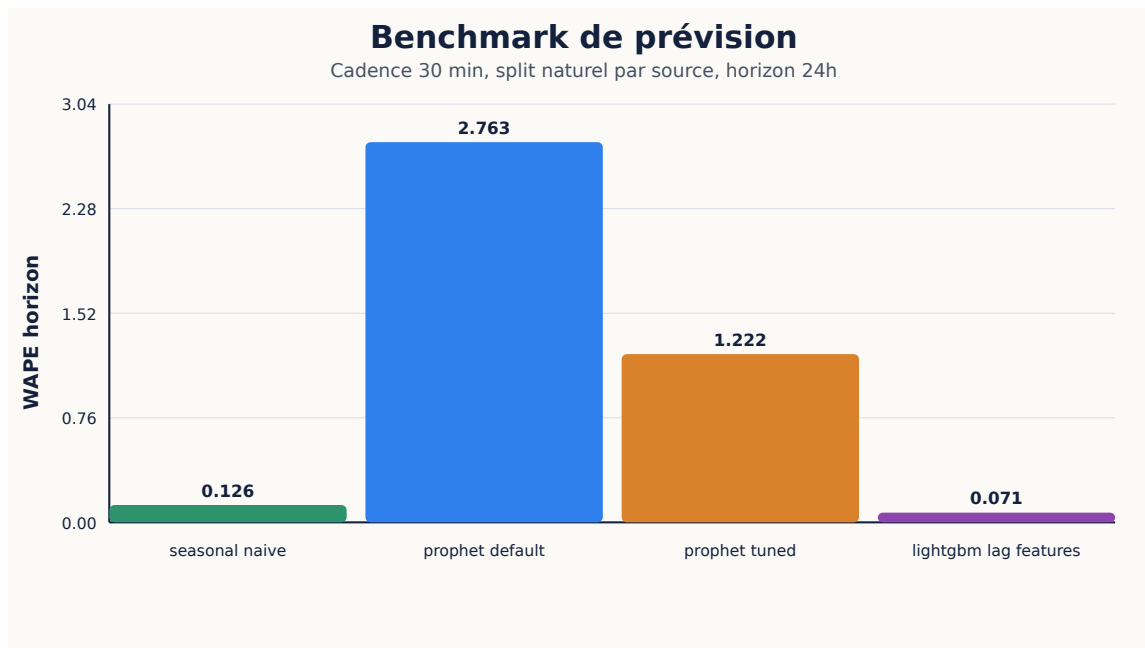


Figure 9: Comparaison WAPE sur horizon 24h. LightGBM obtient la meilleure précision brute sur ce jeu synthétique; Prophet reste le modèle le plus explicable pour raconter tendance et saisonnalité.

## Prophet et LSTM Autoencoder

Prophet modélise une série comme somme de composantes (tendance, saisonnalités, effets calendaires) et sert de baseline interprétable. Le LSTM Autoencoder apprend à reconstruire des fenêtres normales et signale les reconstructions coûteuses; il reste une baseline de recherche (F1 ligne 0.1933).

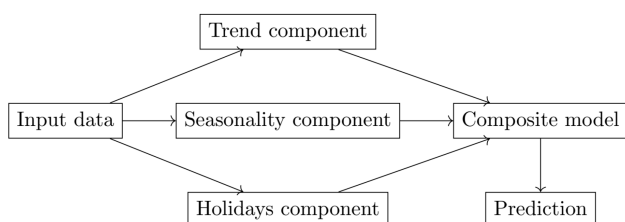


Figure 10: Principe Prophet: tendance + saisonnalités.

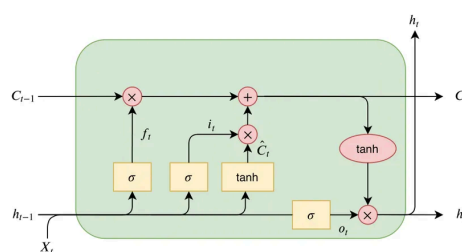


Figure 11: Architecture LSTM: portes mémoire / oubli / sortie.

# Détection d'anomalies et métriques

La détection compare la consommation réelle à la consommation prévue. Le signal utile est le résidu  $|\text{réel} - \hat{y}|$ ; une alerte est créée lorsque ce résidu dépasse un seuil robuste médiane/MAD. Le scoring distingue lignes individuelles et événements opérationnels: un événement non ponctuel doit accumuler  $k = 5$  flags, les pics et drops ponctuels gardent  $k = 1$ .

Meilleur WAPE 24h

**0.0707**

F1 ligne

**0.4068**

F1 événement

**0.5000**

Précision événement

**0.8571**

Les anomalies injectées sont des labels synthétiques vrais, répartis en quatre familles: `point_spike`, `point_drop`, `contextual_day_night_swap` et `trend_drift`. Le seuil MAD retenu par balayage est 4.0.

| Détecteur           | Précision ligne | Rappel ligne | F1 ligne | TP/FP/FN/TN    |
|---------------------|-----------------|--------------|----------|----------------|
| SeasonalNaive + MAD | 0.3731          | 0.3247       | 0.3472   | 25/42/52/841   |
| Prophet tuned + MAD | 0.5102          | 0.3247       | 0.3968   | 25/24/52/859   |
| LightGBM + MAD      | 0.6316          | 0.3000       | 0.4068   | 24/14/56/866   |
| LSTM Autoencoder    | 0.1394          | 0.3152       | 0.1933   | 29/179/63/3761 |

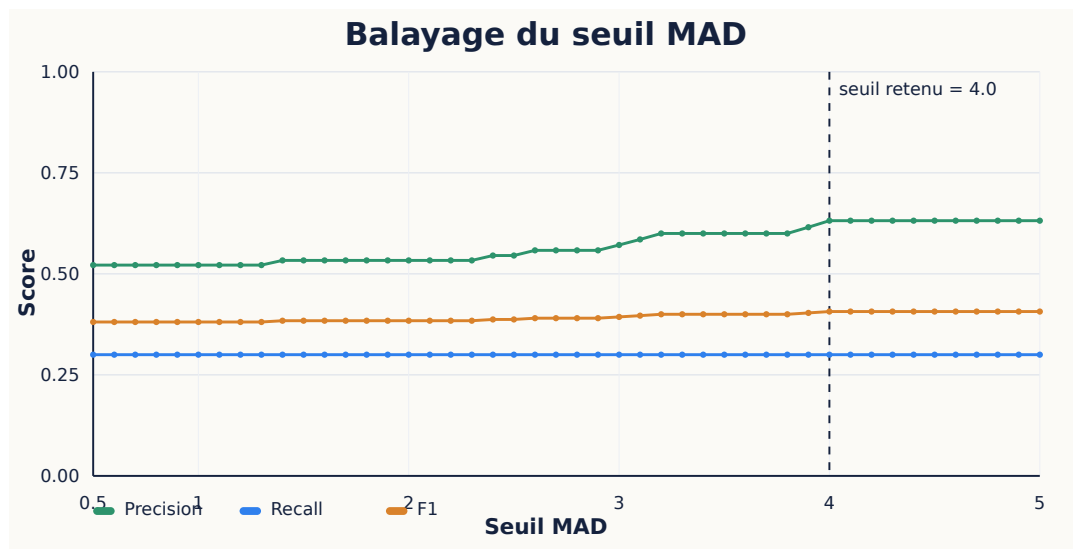


Figure 12: Balayage du seuil MAD sur les résidus LightGBM. Le seuil 4.0 est retenu pour préserver une précision raisonnable tout en gardant des alertes exploitables.

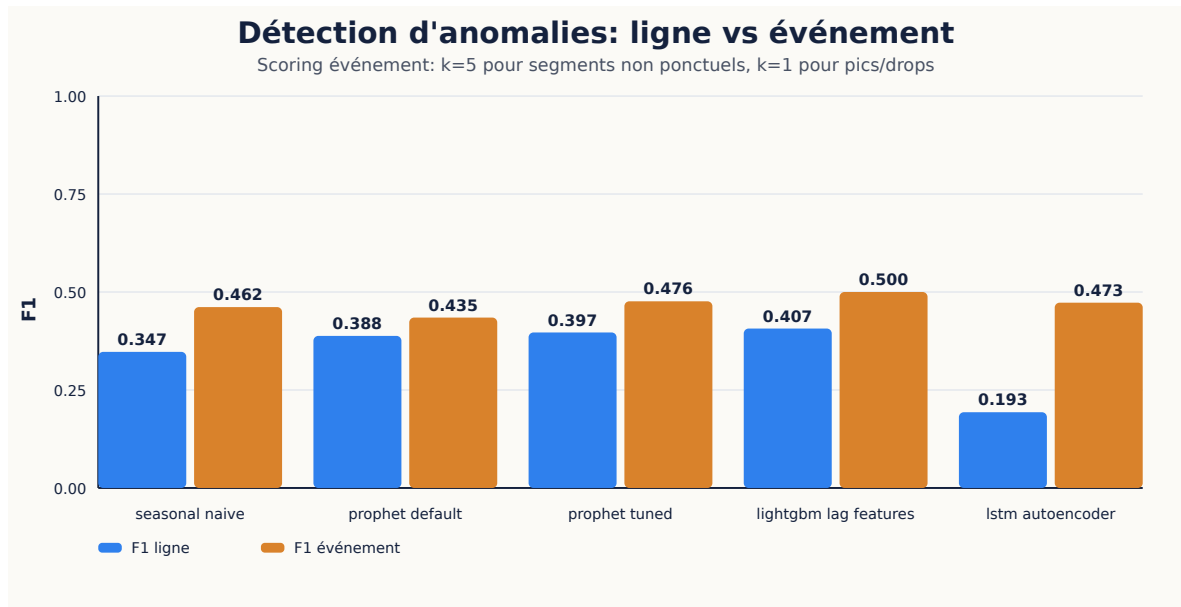


Figure 13: F1 ligne vs F1 événement. Le scoring événement pénalise moins les micro-fluctuations isolées, mais montre que les anomalies contextuelles et les drifts graduels restent difficiles pour un simple résidu MAD.

# Ingestion, stockage et monitoring

La démonstration locale lance Kafka, Kafka UI, TimescaleDB, Prometheus, Grafana et les exporters. Le topic `smartgrid.meters.raw` est alimenté, les lectures et sorties ML sont chargées dans l'entrepôt, les agrégats rafraîchis et les services vérifiés.

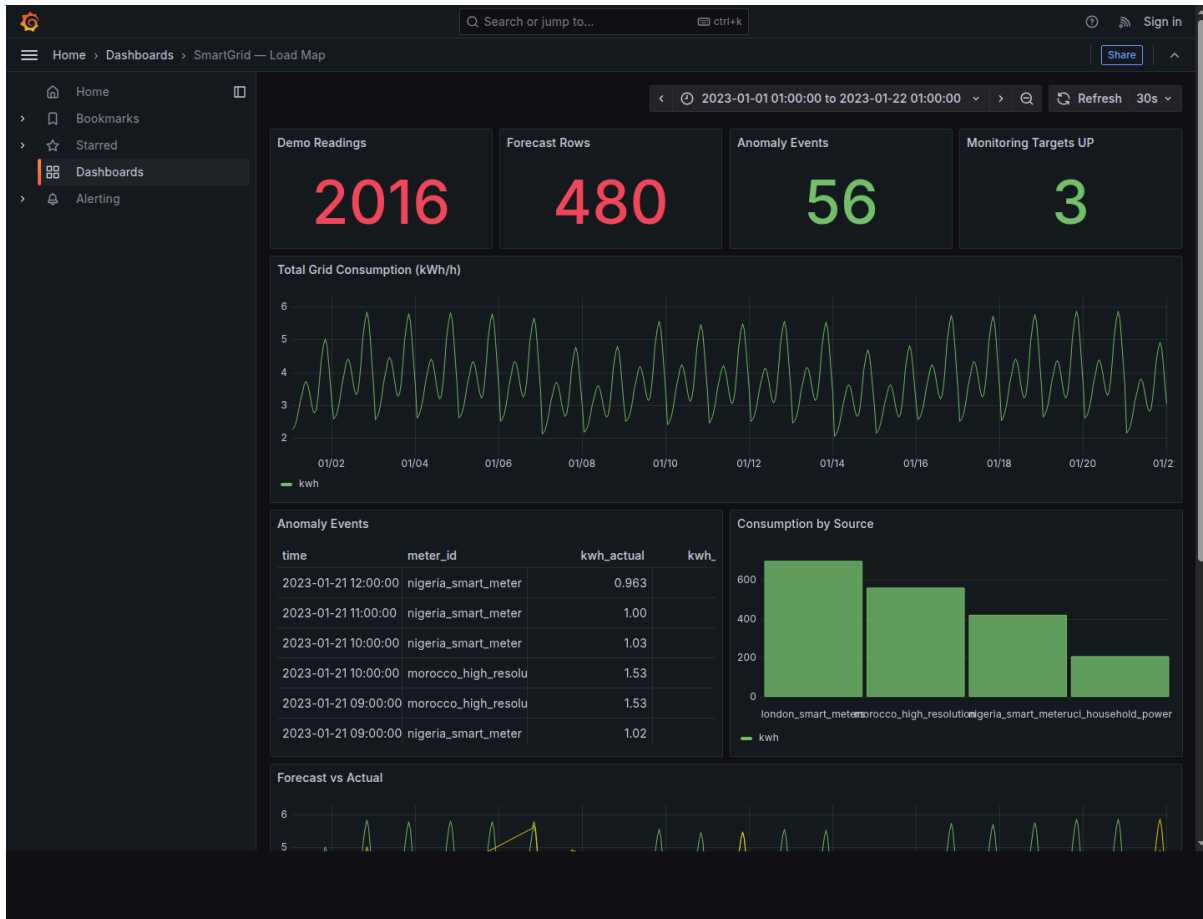


Figure 14: Grafana: dashboard SmartGrid peuplé — lectures, prévisions, anomalies, consommation par source et indicateur de monitoring.

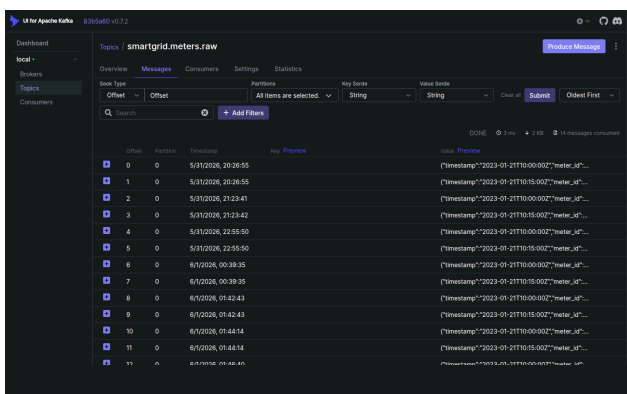


Figure 15: Kafka UI: topic `smartgrid.meters.raw` avec messages live.

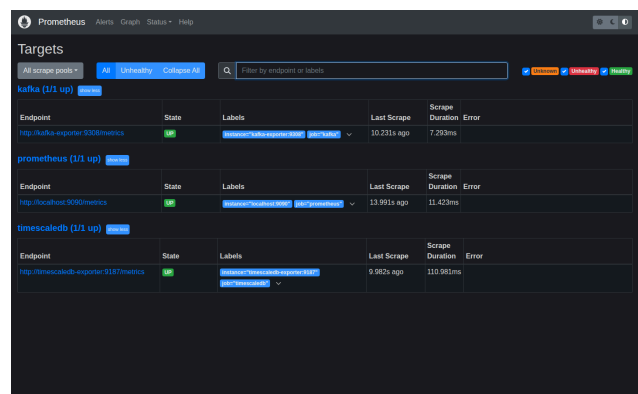


Figure 16: Prometheus: targets Prometheus / Kafka / TimescaleDB en état UP.

# Conclusion

Le projet valide une chaîne SmartGrid cohérente de bout en bout. La partie entrepôt fournit un modèle dimensionnel solide — faits temporels et dimensions conformes — adapté à une charge analytique et déployable sans interruption. La partie IA repose sur une idée simple et vérifiable: une anomalie est un écart significatif entre la consommation observée et la consommation prévue pour la même source. Les deux contributions se rejoignent dans le même entrepôt et le même tableau de bord, de l'ingestion jusqu'à la décision.